

Sustainable transformation through innovation"

SCHOOL OF SCIENCE AND TECHOLOGY

BACHELOR OF COMPUTER SCIENCE

Group 1: Vincent Mugenzi, Lewis, Tino, Timmy, Lumpombwe
YEAR OF STUDY: Year 2
SEMESTER OF STUDY: Semester 2
COURSE NAME & CODE: WEB PROGRAMMING 2 BSc. Comp 222
ASSIGNMENT: Assignment 2
DUE DATE: 9th October 2026
COURSE LECTURER: Mr. Mwale

Student Records Management System (LGU Manage): Project Report

Second Semester 2026, Web Programming 2, PHP Object-Oriented Programming

1. Introduction and architecture

LGU Manage is a PHP 8 web application that lets an administrator register students, enrol them in courses, record marks, and view transcripts, GPAs and reports (Project 1 of the brief). Data is stored in MySQL through PDO, which covers the stretch goal, and GPA is weighted by credit hours. The code follows the Model–View–Controller pattern. *public/index.php* is the only entry point; a *Router* maps each URL to a controller action; controllers use repositories to load and save objects; and plain-PHP views display the result. Business rules such as the 0–100 mark range and the duplicate-enrolment check live inside the domain classes, so no page script can bypass them. Classes are loaded by a PSR-4 autoloader, one class per file, with the domain classes in the classes/ folder.

2. Class diagram and responsibilities

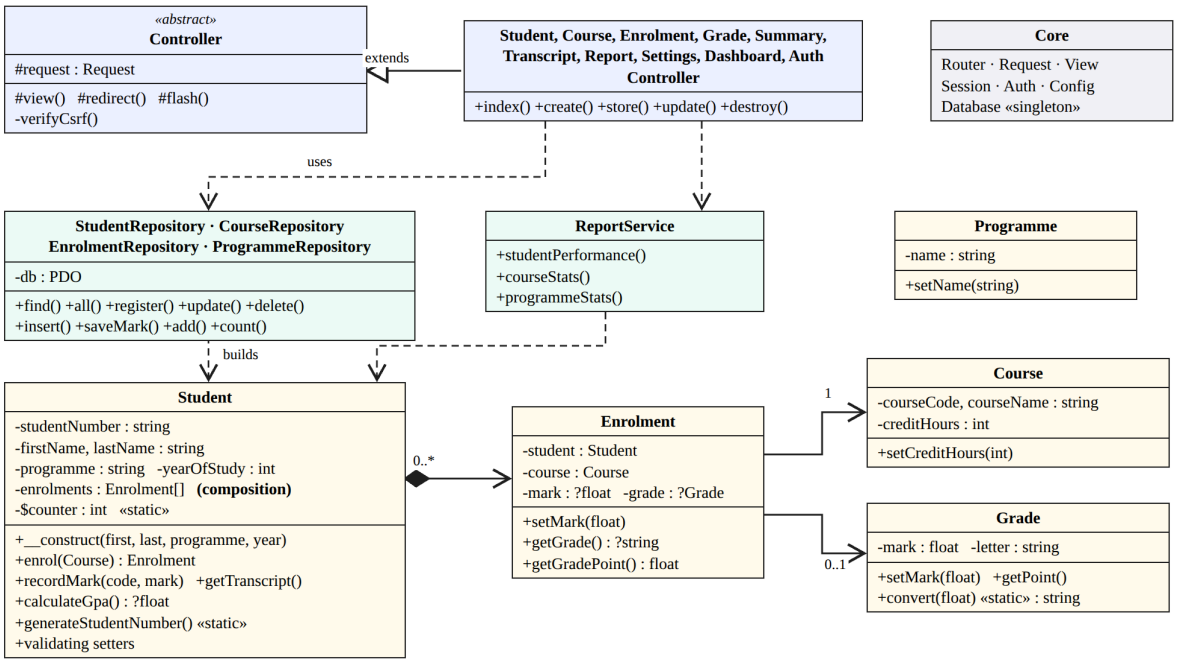


Figure 1: Class diagram (solid diamond = composition; dashed arrow = uses / creates)

Class	Layer	Responsibility
Student	Classes	Identity (auto-numbered), programme and year; owns its Enrolments; enrol(), recordMark(), getTranscript(), calculateGpa().
Course	Classes	Course code, name and credit hours (validated 1–12).
Enrolment	Classes	Links one Student to one Course; stores the mark and exposes getGrade() and grade points.
Grade	Classes	Validates a mark (0–100) and converts it to A, B+, B, C+, C, D or F; supplies grade points for GPA.
Programme	Classes	A validated degree programme name used when registering students.
Student-, Course-, Enrolment-, ProgrammeRepository	Repository	All SQL (PDO prepared statements); turns database rows into objects and saves them back.
ReportService	Service	Builds performance, course and programme statistics from the domain objects.
Controller (abstract) and 10 subclasses	Controller	Read input, call the classes and repositories, flash messages, redirect after POST; the base class verifies the CSRF token.
Router, Request, View, Session, Auth, Config, Database	Core	Front-controller plumbing: URL matching and login check, template rendering, sessions, hashed admin login, one shared PDO connection.

Class	Layer	Responsibility
Views (2 layouts, 12 page folders)	View	Presentation only; every value is escaped with htmlspecialchars().

3. Where each OOP concept is used

Concept	Where	How it is applied
Classes and objects	Student, Course, Enrolment, Grade	Each record is an object; repositories build the objects from joined database rows.
Encapsulation	All classes	Private properties with getters and setters; validation sits in the setters (Grade::setMark rejects marks outside 0–100, Course::setCreditHours 1–12, Student::setYearOfStudy 1–6).
Constructors	Student::__construct()	Takes names, programme and year, validates them through the setters, and generates a student number when none is supplied.
Composition	Student holds Enrolment[]	enrol() creates the Enrolment inside the Student; an Enrolment cannot exist without its Student and Course.
Static members	Student::$counter, generateStudentNumber(), Grade::convert()	The static counter yields LGU-2026-001, -002 and so on; the repository seeds it from the database so numbers never restart.
Inheritance, abstract class	Controller	Shared behaviour (render, redirect, flash, CSRF check) is written once; 10 controllers extend it.
Polymorphism, interface	Router; Grade implements Stringable	The Router creates any Controller subclass by name and calls its action through the common type; Grade::__toString() returns the letter grade.
Exceptions	Classes, controllers, index.php	The classes throw InvalidArgumentException (bad mark, duplicate enrolment) or RuntimeException (not enrolled); controllers catch and show a message; index.php catches Throwable, so no raw PHP error is shown.
Type hinting	Every file	declare(strict_types=1), typed parameters and returns, nullable (?float) and never types.
Design patterns, security	Repositories, Database, forms	Repository and data-mapper pattern, singleton connection, front controller; prepared statements, CSRF tokens, password_hash(), output escaping.

4. Screenshots of the running application

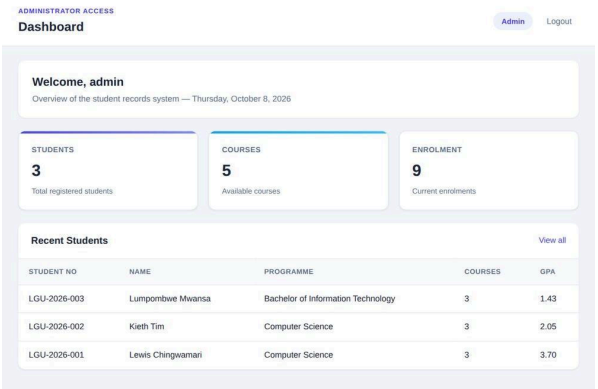


Figure 2: Dashboard with totals and recent students

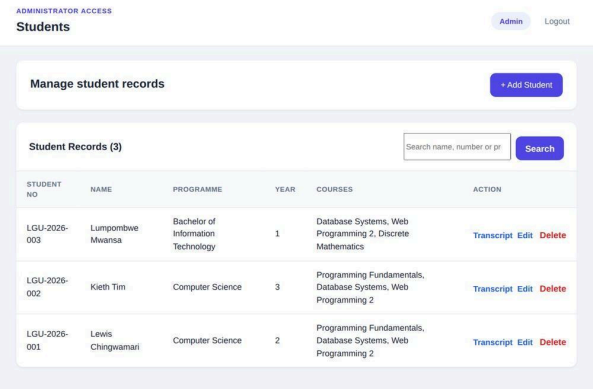


Figure 3: Student list with search, edit, delete and transcript

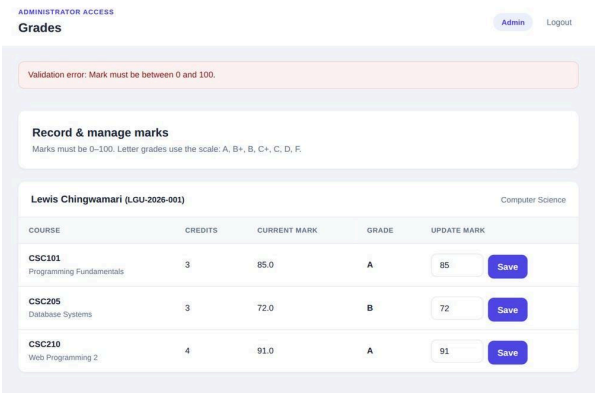


Figure 4: Exception caught: mark 120 rejected (0–100 rule)

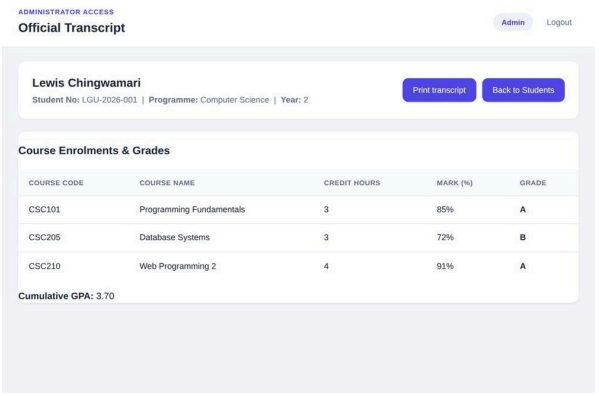


Figure 5: Transcript with letter grades and weighted GPA

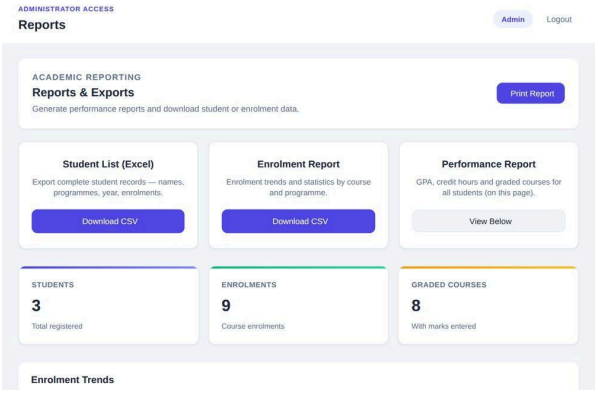


Figure 6: Reports with CSV export

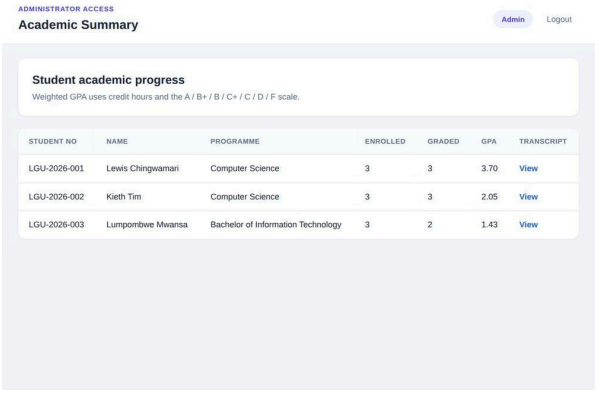


Figure 7: Academic summary with weighted GPA per student

5. Error handling and input validation

Input is checked in two places. The HTML forms use required, min and max attributes as a convenience, but the real rules sit in the class setters, so they apply however a value arrives. A bad value throws InvalidArgumentException: Grade::setMark() rejects any mark outside 0–100 (the case shown in Figure 4), Course::setCreditHours() allows 1–12, Student::setYearOfStudy() allows 1–6, and empty names or programmes are refused. Student::enrol() rejects a duplicate enrolment, and recordMark() throws RuntimeException if the student is not enrolled in that course.

Every controller action wraps these calls in try/catch, turns the exception into a message for the user and redirects (post-redirect-get), so a raw PHP error is never shown. index.php also catches any other Throwable and shows a generic error page, while database errors are written to the log instead of the screen. All SQL uses PDO prepared statements, every output is escaped with htmlspecialchars(), and each POST form carries a CSRF token that the base Controller checks before any action runs; a missing or wrong token is refused with HTTP 419.

AI acknowledgement: Claude (Anthropic) was used to help restructure the code into the MVC layout and draft this report. The author is responsible for the content and can explain the code.

References

Fowler, M. (2002). *Patterns of enterprise application architecture*. Addison-Wesley.

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley.

Martin, R. C. (2017). *Clean architecture: A craftsman's guide to software structure and design*. Prentice Hall.

Meyer, B. (1997). *Object-oriented software construction* (2nd ed.). Prentice Hall.

OWASP Foundation. (n.d.). *Cross-site request forgery prevention cheat sheet*. https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html

PHP Documentation Group. (n.d.). *PHP manual: Classes and objects*. https://www.php.net/manual/en/language.oop5.php

PHP-FIG. (n.d.). *PSR-4: Autoloader*. https://www.php-fig.org/psr/psr-4/